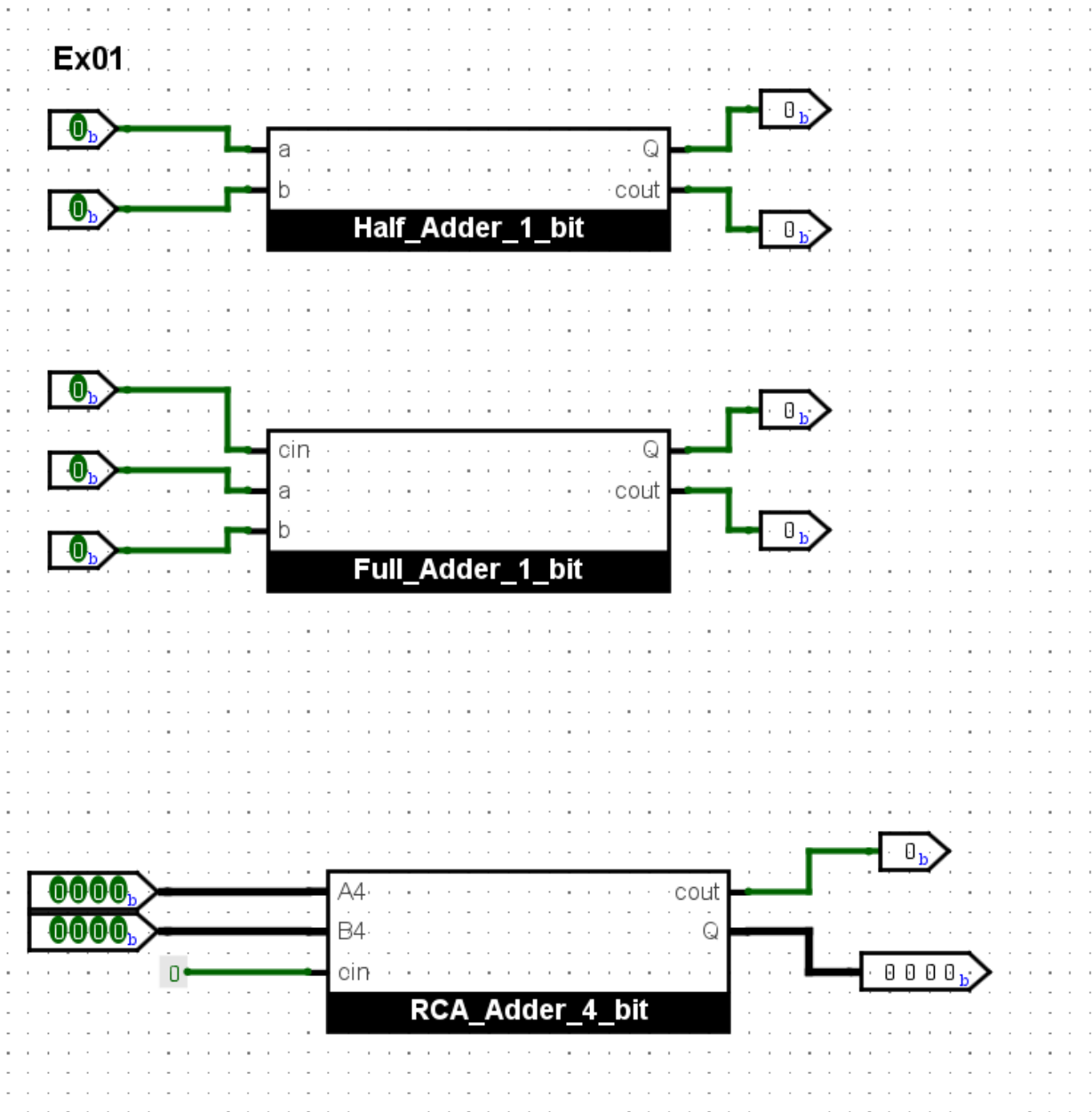


Full name: Doan Cong Thanh

Class: 22TT2

Id: 22125096

Ex01:



Half Adder 1 bit: we have 2 inputs (1 bit) with c value equals to 0.

We have the following truth table for the sum of these 2 data.

a	b	Q	cout
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

And we have the function:

$$Q = a \oplus b$$

$$\text{cout} = a \cdot b$$

Full Adder 1 bit: we have 2 inputs (1 bit) with cin value (1 bit).

We have the following truth table for the sum of these 2 data.

cin	a	b	Q	cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

K-map (Q):

		a, b			
		00	01	11	10
cin	0	0	1	0	1
	1	1	0	1	0

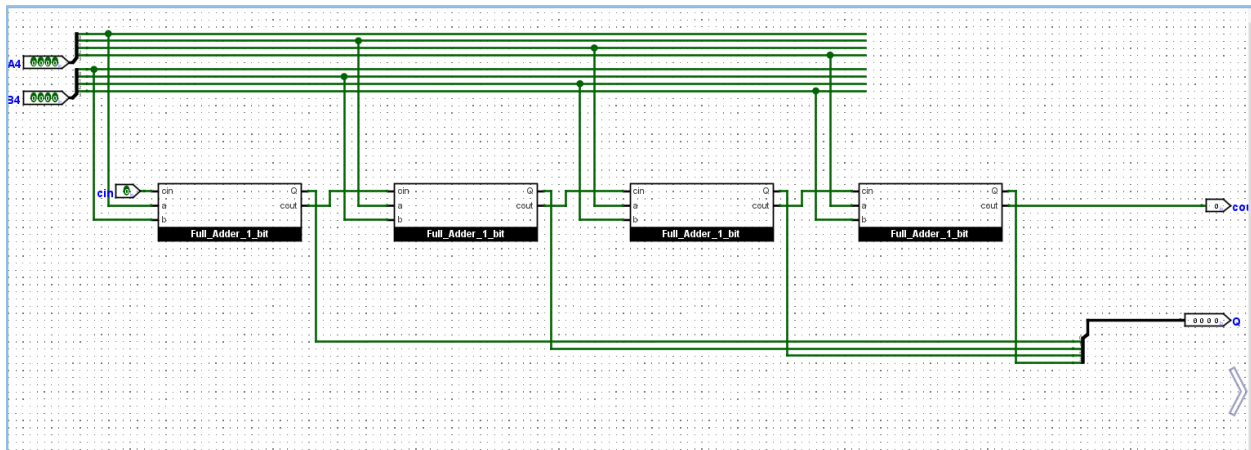
$$Q = \sim \text{cin} \cdot \sim a \cdot b + \sim \text{cin} \cdot a \cdot \sim b + \text{cin} \cdot \sim a \cdot \sim b + \text{cin} \cdot a \cdot b$$

K-map(cout):

a, b		cin			
		00	01	11	10
cin	0	0	0	1	0
	1	0	1	1	1

$$\text{cout} = a \cdot b + \text{cin} \cdot b + \text{cin} \cdot a$$

RCA Adder 4 bit:



To sum 2 input (more than 1 bit), we use the RCA (Ripple Carry Adder) method.

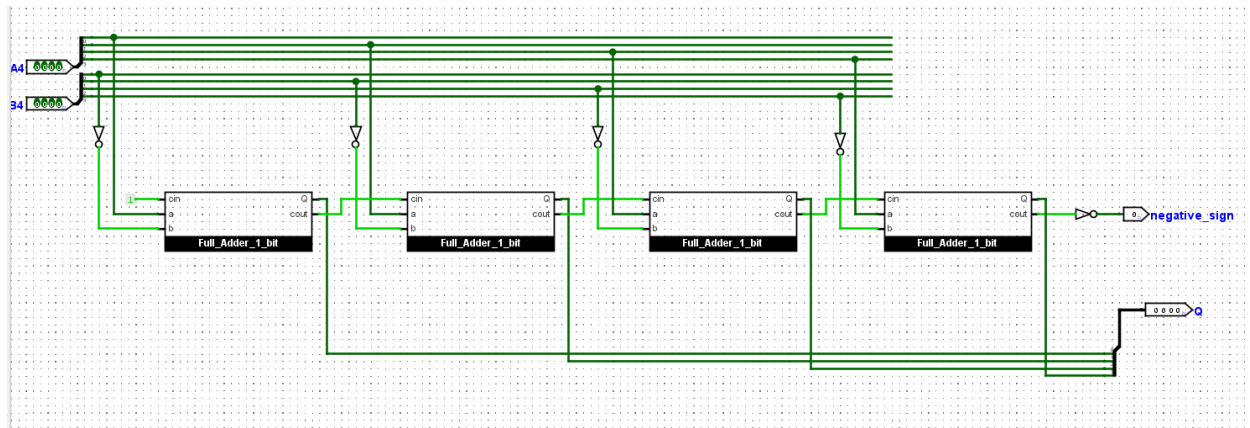
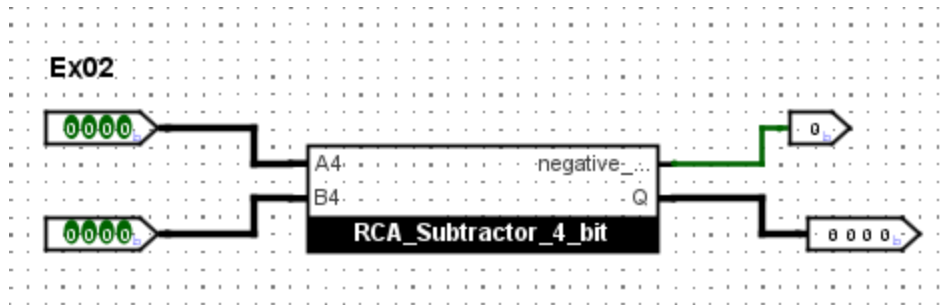
Assume:

A4(a0a1a2a3) and B4(b0b1b2b3). We use the first full adder 1 bit with cin equals to 0 to sum a0 and b0.

Then use cout_i as cin_{i+1} to sum a_{i+1} and b_{i+1} (with $i = 0, 1, 2$). Repeated this procedure until we add the last two bits.

The result is in format of 4 bit(Q0Q1Q2Q3Q4)

Ex02:



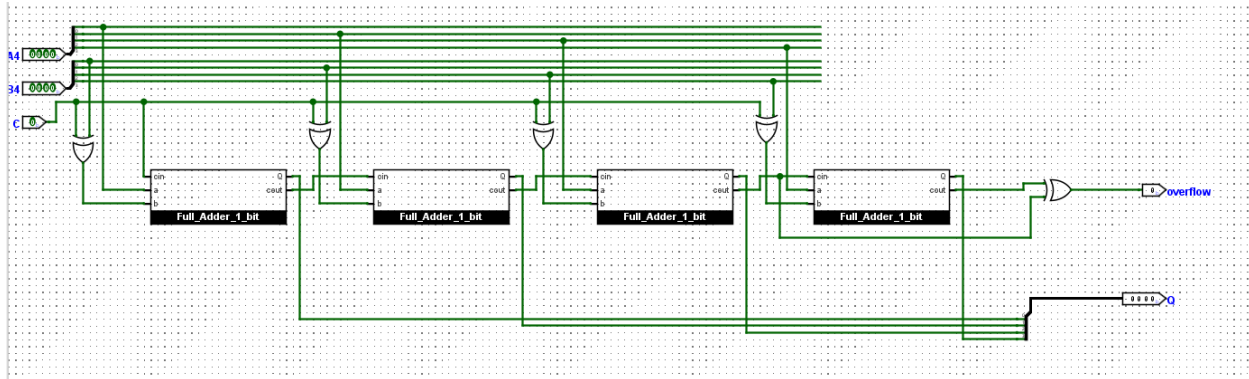
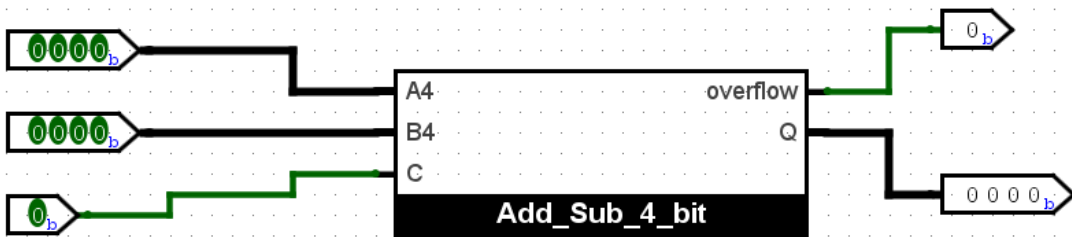
To compute $A - B$. We make the use of 2's complement. So $A - B = A + \sim B + 1$.

So we use the not gate before any bits in B4. Then set cin equals to constant 1.

As we can see, the method is similar to the one explained in Ex01. The only difference is that the $\text{negative_sign} = \sim \text{cout}$. The reason is that in sum operation, when the last cout carry is 1, we detect the overflow. The opposite condition can be seen when we manipulate the subtraction, which means the carry out is 0. So we negate the cout to get correct negative_sign .

Ex03:

Ex03



We use the same idea applied to build the RCA Adder 4 bit with some modification:

To handle the addition/subtraction operation, we use the c value which manage to flip B ($c \text{ XOR } b_i (i = 0, 1, 2, 3)$)

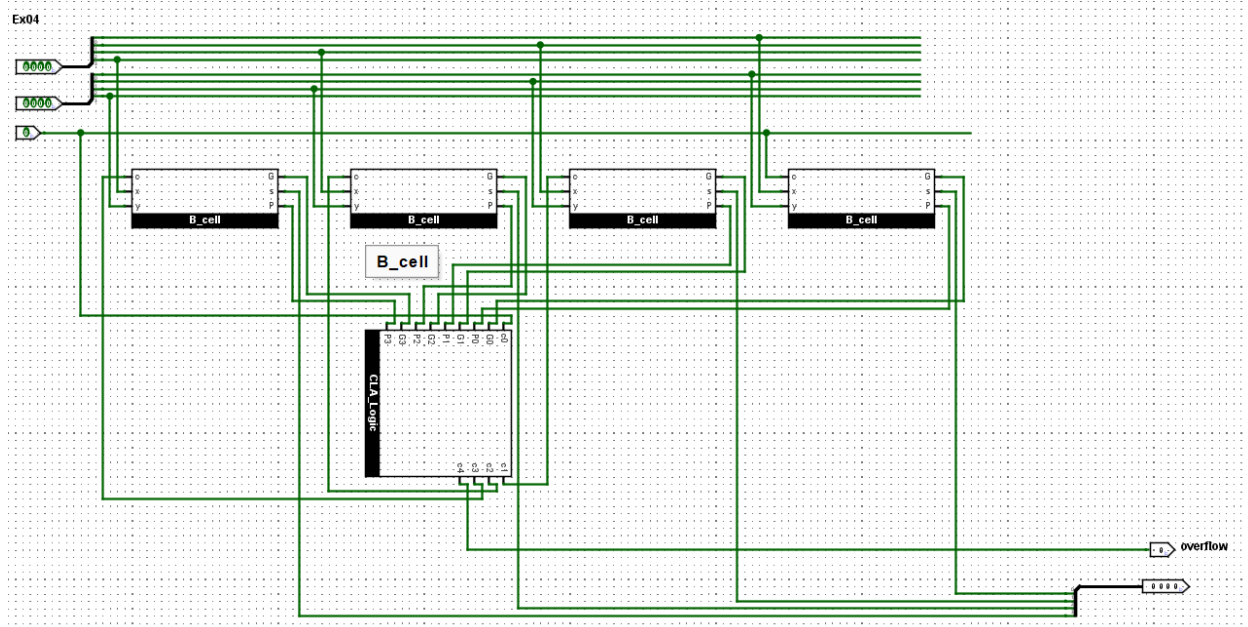
When $c = 1$ (B negates, $c_{in} = 1$)

When $c = 0$ (B remains unchanged, $c_{in} = 0$)

Overflow occurs when carry-in to MSB is different from carry-out of MSB.

$$\text{Overflow} = c_{n-1} \oplus c_n$$

Ex04:



Recall some equation:

$$C_{i+1} = x_i \cdot y_i + x_i \cdot C_i + y_i \cdot C_i = x_i \cdot y_i + (x_i + y_i) \cdot C_i$$

$$Q_i = x_i \oplus y_i \oplus C_i$$

Let denote $G_i = x_i \cdot y_i$, $P_i = x_i + y_i$

We can write $c_{i+1} = G_i + P_i C_i$

The general formula for c_{i+1} depend only c_0 :

$$c_{i+1} = G_i + P_i c_i$$

$$c_i = G_{i-1} + P_{i-1} c_{i-1}$$

$$\Rightarrow c_{i+1} = G_i + P_i (G_{i-1} + P_{i-1} c_{i-1})$$

continuing

$$\Rightarrow c_{i+1} = G_i + P_i (G_{i-1} + P_{i-1} (G_{i-2} + P_{i-2} c_{i-2}))$$

until

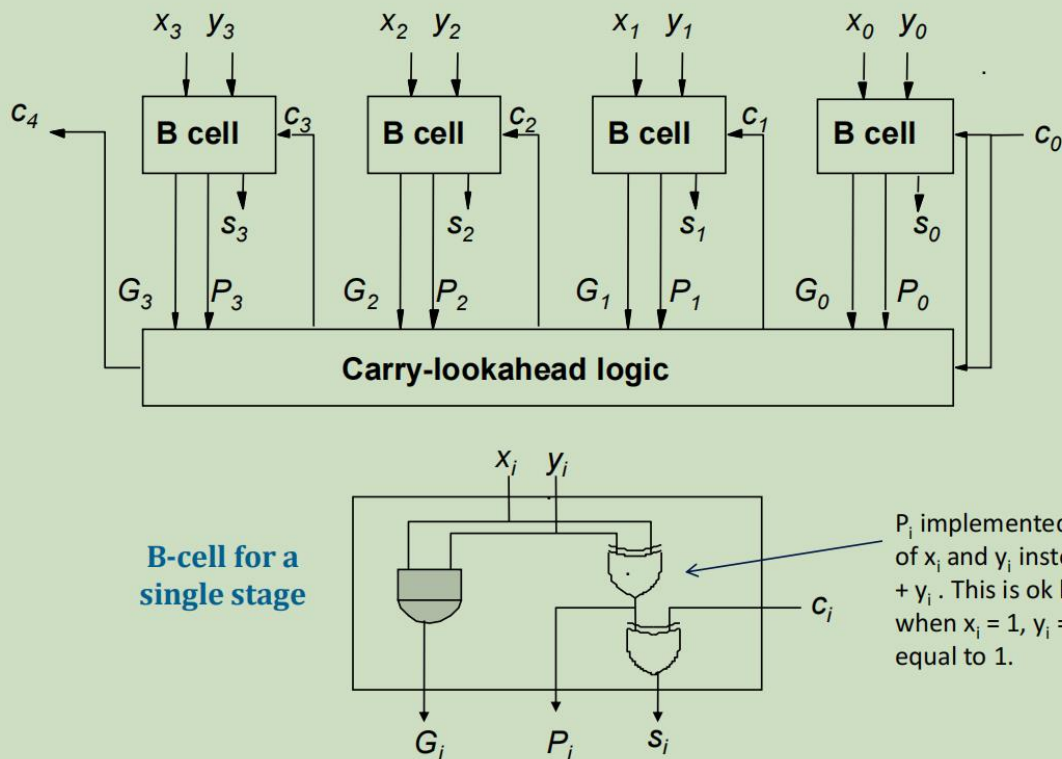
$$c_{i+1} = G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + \dots + P_i P_{i-1} \dots P_1 G_0 + P_i P_{i-1} \dots P_0 c_0$$

By the expectation, when we want to compute c_{i+1} , the efficient way is:

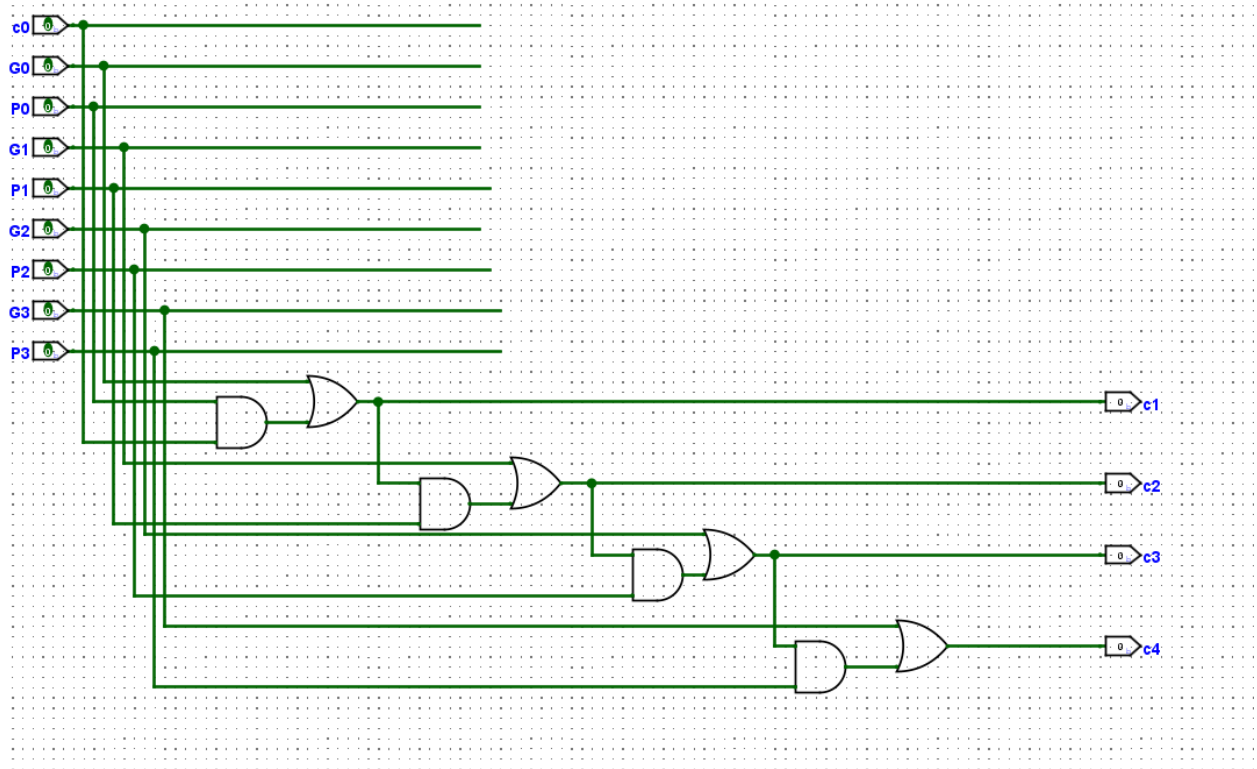
$$C_{i+1} = (C_i \text{ AND } P_i) \text{ OR } G_i \text{ with } (i=1,2,3,\dots)$$

The idea to build CLA following this picture:

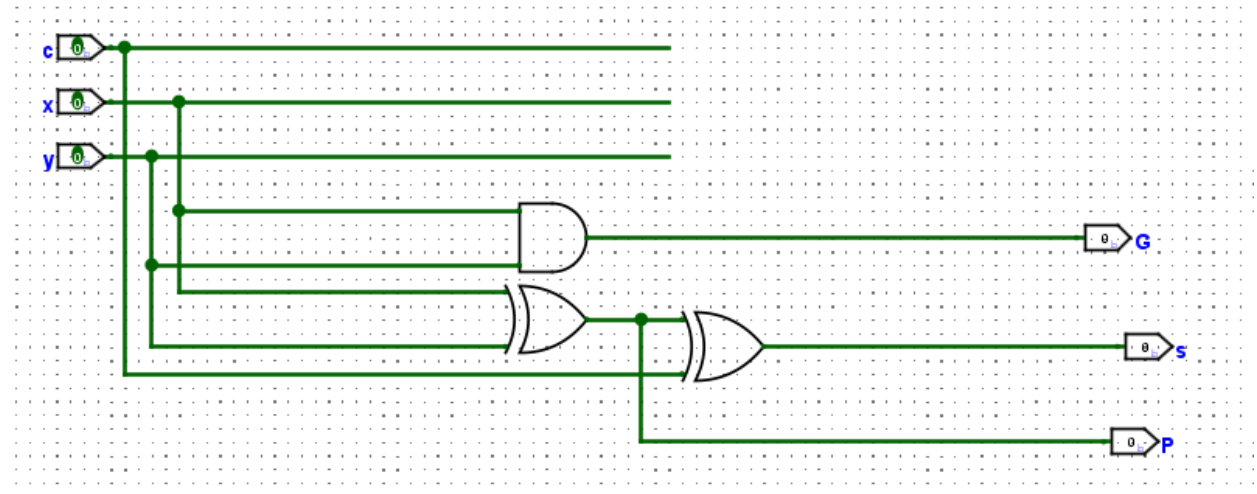
4-bit Carry Lookahead adder



To minimize the cost, we can treat $X \text{ OR } Y$ equivalent to $X \text{ XOR } Y$ since the truth table of these two expressions is quite similar. The only one difference when $x_i = 1, y_i = 1$ do not affect the answer c_{i+1} since G_i is still 1 in this situation, and so $c_{i+1} = (c_i \text{ AND } P_i) \text{ OR } G_i$ with $(i=1,2,3,\dots) = 1$.

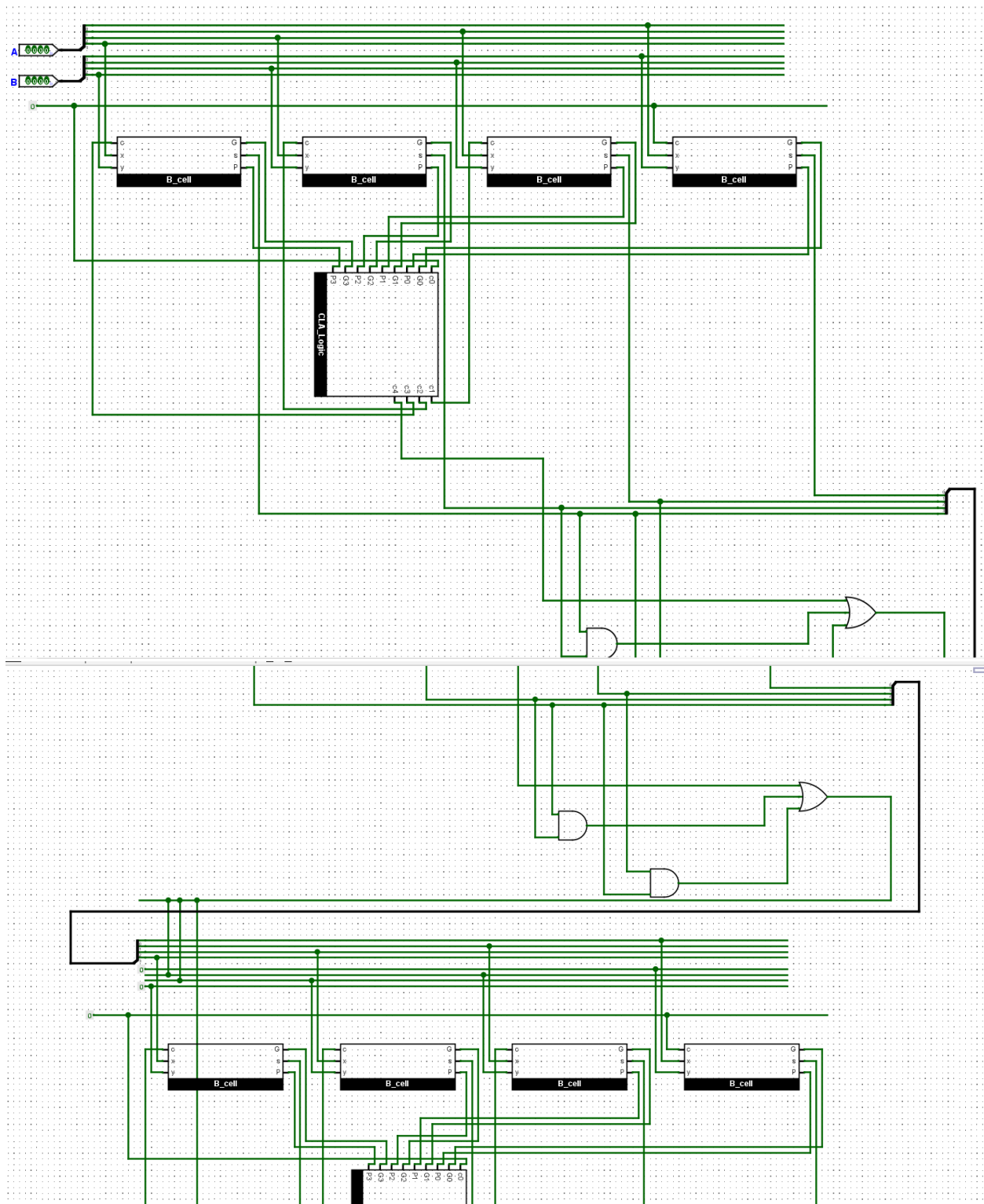


The picture above describes the way I compute c1,c2,c3,c4.



And the last one shows how S,P,G are calculated.

Ex05:



The key idea to build the BCD adder is make the most use of the CLA – Adder to handle the operation fast and efficient.

The way to convert Binary form to BCD form is check the Binary form of the result when adding two number. If the value is greater than 9, the adding procedure, again, will manipulate to increase the first output up to 6 units. Then showing the BCD result in the format: BCD_Output Q3 Q2 Q2 Q0.

To check whether the value is greater than 9 or not, we build the truth table

c	Q3	Q2	Q1	Q0	Condition
0	0	0	0	0	0
0	0	0	0	1	0
0	0	0	1	0	0
0	0	0	1	1	0
0	0	1	0	0	0
0	0	1	0	1	0
0	0	1	1	0	0
0	0	1	1	1	0
0	1	0	0	0	0
0	1	0	0	1	0
0	1	0	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	1
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	1
1	0	1	0	0	-
1	0	1	0	1	-
1	0	1	1	0	-
1	0	1	1	1	-
1	1	0	0	0	-
1	1	0	0	1	-
1	1	0	1	0	-
1	1	0	1	1	-
1	1	1	0	0	-
1	1	1	0	1	-
1	1	1	1	0	-
1	1	1	1	1	-

K-map (Condition):

Q2, Q1, Q0		000	001	011	010	110	111	101	100
c, Q3	00	0	0	0	0	0	0	0	0
	01	0	0	1	1	1	1	1	1
	11	1	1	1	1	1	1	1	1
	10	1	1	1	1	1	1	1	1

$$\text{Condition} = c + Q3 \cdot Q1 + Q3 \cdot Q2$$

So I will set up for the second addition:

A = first result

B (0 Condition Condition 0) 4 bits's data

$$\text{BCD_Sum} = A + B$$

$$\text{BCD_Output} = \text{Condition}$$